

# Rapport for Oblig 1 IN2031

Dino Hukanovic (dino@ifi.uio.no)

Markus André Johnsen ([markusaj@ifi.uio.no](mailto:markusaj@ifi.uio.no))

11 September 2025

## 1) Introduksjon

Denne rapporten dokumenterer løsningen på Oblig1, som gikk ut på å implementere et lite programmeringsspråk kalt AeroScript ved bruk av ANTLR4 for parsing og Java for runtime. Språket støtter aritmetiske operasjoner (addisjon, subtraksjon, multiplikasjon, negasjon), opprettelse av punkter, generering av tilfeldige tall innenfor ranges, samt print-statements.

## 2) Implementering

**Grammatikk (AeroScript.g4):** Definerer språkets syntaks, inkludert tall (heltall og desimaltall), aritmetiske operasjoner (+, -, \*, --), tilfeldige intervaller (*random [min, max]*), opprettelse av punkter (*point x y*), utskriftssetninger, parenteser for gruppering, samt både single-line (*//*) og multi-line (*/\* \*/*) kommentarer. Grammatikk bruker ANTLR4s visitor pattern for tydelig separasjon mellom parsing og evaluation.

**AST og typer:** Det abstrakte syntakstreet bruker et hierarki av node-klasser (*Node*, *NumberNode*, *OperationNode*), der *OperationNode* implementerer ett strategi mønster for ulike operasjoner. Egendefinerte typer inkluderer *Point* for todimensjonale koordinater og *Range* for generering av tilfeldige tall med validering.

**Interpreter:** En visitor-basert evaluator (*Interpreter* utvider *AeroScriptBaseVisitor*) som håndterer evaluering av uttrykk, omfattende type-checking og error handling. Interpreteren validerer operander, sikrer at intervallet er gyldig ( $\min \leq \max$ ), og gir detaljerte feilmeldinger.

Error handling er implementert gjennom en egen *AeroScriptRuntimeException*-klasse med korrekt videresending av feil og informative meldinger. Prosjektet bruker Gradle med ANTLR4-plugin for kodegenerering og JUnit for testing.

### 3) Testing

Testpakken inkluderer enhetstester ved bruk av JUnit, som dekker:

- Grunnleggende aritmetiske operasjoner (*InterpreterTest*)
- Individuelle AST-nodeoperasjoner (*OperationNodeTest*)
- Generering av tilfeldige tall innenfor gyldige intervaller
- Opprettelse av punkt med ulike koordinatverdier

Testene demonstrerer korrekt evaluering av uttrykk som  $2 + 3$ ,  $--3$  og *random* [4, 10].

#### 3.1) Begrensninger ved testene

De automatiserte testene dekker primært scenarier med korrekt input. Selv om implementasjonen inkluderer omfattende error handling for ugyldige intervaller, type-mismatches og null-operander, er disse feiltilfellene ikke dekket i særlig grad i den automatiserte testpakken, men kan valideres gjennom manuell testing.

### 4) Begrensninger og potensielle problemer

- **Statisk vs. runtime typetesting:** Kun runtime typetesting er støttet; ingen statisk typeanalyse utføres under parse-tid.
- **Ikke-deterministisk testing:** Generering av tilfeldige tall gjør at noen testresultater blir ikke-deterministiske, selv om validering av intervaller sikrer korrekthet.
- **Begrenset testdekning:** Testpakken kunne utvides til å inkludere flere grensetilfeller og feilscenarier.

### 5) Konklusjon

Implementasjonen oppfyller alle oppgavekrav med en ryddig og utvidbar arkitektur. Språket støtter alle spesifiserte funksjoner, inkludert aritmetiske operasjoner, opprettelse av punkter, generering av tilfeldige tall og print-statements. Bruken av visitor pattern gir en god separasjon av ansvar, og den omfattende error handling gjør tolken robust. Til tross for begrenset testdekning viser løsningen god innsikt i språkimplementasjon.